

# CONTENTS

## Práctico 5 (Circuitos Secuenciales — Flip-Flops, Contadores, Máquinas de Estado) — Arquitectura de Computadoras

|                                                                                                          |       |
|----------------------------------------------------------------------------------------------------------|-------|
| FING UdelaR · Resolución completa y verificada                                                           | ..... |
| Preguntas teóricas                                                                                       | ..... |
| Ejercicio 1 ★★ — Contador de 4 bits up/down                                                              | ..... |
| Ejercicio 2 ★ — Verificador secuencial de paridad                                                        | ..... |
| Ejercicio 3 ★ — Comparador serial de bits                                                                | ..... |
| Ejercicio 4 ★★ — Contador de código Gray de 3 bits                                                       | ..... |
| Ejercicio 5 ★★ — Sumador serial de 8 bits                                                                | ..... |
| Ejercicio 6 ★ (OpenFing, sin solución en el material del curso) — Contador de números primos             | ..... |
| Ejercicio 7 ★★★ (examen 26/07/2014, sin solución en el material del curso) — Cronómetro minutos:segundos | ..... |
| Paso 1 — Descomposición en 4 contadores en cascada                                                       | ..... |
| Paso 2 — Lógica de acarreo entre etapas (contador normal, INCR=0)                                        | ..... |
| Paso 3 — Botón incrementar (INCR)                                                                        | ..... |
| Paso 4 — Reset                                                                                           | ..... |
| Paso 5 — Displays de 7 segmentos                                                                         | ..... |
| Resumen de técnicas usadas en este práctico                                                              | ..... |

# Práctico 5 (Circuitos Secuenciales — Flip-Flops, Contadores, Máquinas de Estado) — Arquitectura de Computadoras

## FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

**Cómo se verificó este práctico:** todas las tablas de estados/transición y todas las ecuaciones de excitación de este documento fueron generadas y comprobadas con un script de Python ( `sympy` para la minimización booleana por fuerza bruta — equivalente a un Karnaugh exhaustivo — más simulación bit a bit para las máquinas de estado y los contadores en cascada). El material oficial del curso ( [Ejercicios Resueltos.pdf](#) ) solo trae resueltos los ejercicios 1 a 5 y las preguntas teóricas; los Ejercicios 6 y 7 no tienen solución publicada (Ej. 7 es un ejercicio de examen 2014), así que se resolvieron acá desde cero con la misma metodología del curso.

**⚠ Nota de transparencia sobre el reset:** la letra indica como regla general “Todos los circuitos construidos deben incluir reset. A menos que se indique lo contrario, el reset debe ser activo por nivel bajo.” Los Ejercicios 3 y 5 indican explícitamente “reset que activa en 1” (nivel alto) — para esos dos se usó nivel alto. Para el resto (Ej. 1, 2, 4, 6, 7, y el FF R-S/JK de las teóricas) se usó el default de la letra: **reset activo por nivel bajo**, implementado como entrada asincrónica de Clear/Preset de cada Flip-Flop (nunca como variable sincrónica dentro del Karnaugh), siguiendo la teoría del curso (Guía M4, sección 1: “Set/Preset y Clear asincrónicos”). El material de referencia del curso, en las tablas de transición de los Ejercicios 1, 2 y 4, parece haber modelado el reset como sincrónico y activo en 1 (contradiendo el propio enunciado) — se señala la discrepancia pero se resolvió siguiendo la letra tal cual está escrita, que es lo exigible en un parcial/examen real.

## Preguntas teóricas

**(a) Flip-flop sensible a nivel vs. sensible a flanco.** Un FF sensible a **nivel** (Latch, ej. R-S con habilitación G) responde a sus entradas de datos **todo el tiempo** que la señal de control esté en su nivel activo (por ejemplo, mientras  $G=1$ ): cualquier cambio de R/S durante ese intervalo se refleja en la salida. Un FF sensible a **flanco** solo “mira” sus entradas de datos en el instante exacto de la transición de reloj ( $0 \rightarrow 1$  ascendente, o  $1 \rightarrow 0$  descendente) — los cambios de esas entradas en cualquier otro momento del período de reloj no tienen efecto. Es la diferencia entre “ventana de tiempo” (nivel) y “instante puntual” (flanco).

**(b) ¿Una memoria RAM es secuencial o combinacional? ¿Por qué?** Es **secuencial**. La salida de una lectura no depende solo de la entrada actual (la dirección): depende también de la **historia de escrituras** sobre esa misma dirección — si escribo un dato y después lo leo, obtengo lo que escribí, no una función fija de la dirección. Esa dependencia de valores pasados es exactamente la definición de circuito con memoria/estado. Internamente, además, cada celda de una RAM está construida con un Flip-Flop (SRAM) o su equivalente en capacitor (DRAM), que son por definición elementos de memoria.

**(c) Construir un flip-flop JK a partir de flip-flops D.** Método: igualar la ecuación característica del FF deseado (JK) con la del FF disponible (D), y despejar la entrada del FF disponible.

Ecuación característica JK:  $Q_{n+1} = J \cdot Q_n' + K' \cdot Q_n$  Ecuación característica D:  $Q_{n+1} = D_n$

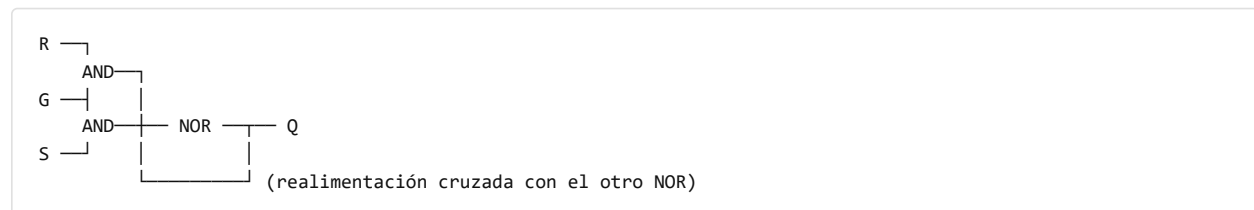
Igualando:  $D = J \cdot Q' + K' \cdot Q$

Basta con armar ese circuito combinacional (2 AND + 1 OR + 2 NOT, realimentando la propia salida Q) y conectarlo a la entrada D de un FF D físico; el resultado se comporta exactamente como un FF JK.

**Verificación (Python, las 8 combinaciones de J,K,Q):** se evaluó  $D = J \cdot Q' + K' \cdot Q$  y se aplicó la regla del FF D ( $Q_{n+1} = D$ ) para las 8 filas, comparando contra la tabla JK original (mantiene/reset/set/invierte). Coincidencia exacta en las 8 filas. Ejemplo de chequeo manual:  $J=K=1, Q=1$  (debe invertir  $\rightarrow Q_{n+1}=0$ ).  $D = 1 \cdot 0 + 0 \cdot 1 = 0 \rightarrow$  FF D copia D  $\rightarrow Q_{n+1}=0 \checkmark$ .

#### (d) Flip-flop R-S: dibujo y por qué es secuencial.

Circuito clásico (dos NOR realimentadas, con habilitación sincrónica G — la versión “R-S sincrónico” que además permite justificar (e)):



Con  $G=1$ : si  $R=1, S=0 \rightarrow$  el AND de arriba fuerza la entrada del NOR superior a 1  $\rightarrow Q=0$ . Si  $S=1, R=0$  (simétrico)  $\rightarrow Q=1$ . Si  $R=S=0$ , **ambos AND dan 0**, y la salida del NOR queda determinada exclusivamente por el valor de Q en el ciclo anterior — es decir,  $Q_{n+1}=Q_n$ .

**Por qué esto demuestra que es secuencial:** con la **misma** entrada  $(R,S)=(0,0)$ , el circuito puede estar en  $Q=0$  o en  $Q=1$  según cuál fue la última entrada distinta de  $(0,0)$  que recibió — el mismo par de entradas produce dos salidas posibles según la historia. Esa es precisamente la firma de un circuito con memoria/estado ( $Y=G(X,E)$ , no  $Y=G(X)$ ).

**(e) ¿Reset y Preset son sincrónicas o asincrónicas? Asincrónicas.** Actúan por nivel y de forma inmediata, sin esperar ningún flanco de reloj — Preset fuerza  $Q=1$  y Clear/Reset fuerza  $Q=0$  en el instante en que se activan, independientemente del estado de CLK.

## Ejercicio 1 ★★ — Contador de 4 bits up/down

**Enunciado:** contador de 4 bits con una entrada adicional UD: si  $UD=1$  incrementa, si  $UD=0$  decrementa.

**Diseño elegido: Flip-Flops tipo T** (más simple de justificar y de construir que D para este caso — la letra solo pide “flip-flops”, sin especificar tipo). Con T, cada bit  $i$  invierte su valor exactamente cuando corresponde “acarrear” (subiendo) o “pedir prestado” (bajando) desde los bits menos significativos:

$$T_0 = 1 \text{ (siempre invierte, es el bit que más rápido cambia)} \quad T_i = UD \cdot (q_{i-1} \cdots q_0) \oplus UD' \cdot (q_{i-1} \cdots q_0)' \quad (i=1,2,3)$$

Es decir,  $T_i = 1$  si (subiendo, todos los bits menos significativos están en 1  $\rightarrow$  hay acarreo) o (bajando, todos están en 0  $\rightarrow$  hay préstamo).

Para 4 bits ( $q_3q_2q_1q_0$ ):

$$\begin{aligned} T_1 &= UD \odot q_0 \oplus (q_0 \text{ XOR } UD) \\ T_2 &= UD \odot q_1 \oplus q_0 + UD \odot q_1' \oplus q_0' \\ T_3 &= UD \odot q_2 \oplus q_1 \oplus q_0 + UD \odot q_2' \oplus q_1' \oplus q_0' \end{aligned}$$

Reset asincrónico (nivel bajo) sobre las 4 líneas de Clear → estado inicial 0000.

**Verificación (Python):** se generaron las 32 combinaciones (estado de 0 a 15,  $UD \in \{0,1\}$ ), se calculó el estado siguiente esperado  $((q+1) \bmod 16$  si  $UD=1$ ,  $(q-1) \bmod 16$  si  $UD=0$ ), y se comparó contra el resultado de aplicar  $T_i = Q_i \oplus T_i$  con las fórmulas de arriba — coincidencia exacta en las 32 filas.

**Verificación cruzada con Flip-Flops D:** se generó independientemente, por minimización SOP exhaustiva (sympy, sin usar las fórmulas de  $T$ ), el conjunto de ecuaciones  $D_i(q_3, q_2, q_1, q_0, UD)$  para las 32 filas — también verificado exacto. Salen equivalentes mediante  $D_i = Q_i \oplus T_i$ , pero mucho más largas de escribir a mano (5 variables por Karnaugh); por eso la forma  $T$  de arriba es la que conviene memorizar y usar en un parcial.

**Circuito:** 4 FF T en cascada. FF0 con  $T_0=1$  fijo. FF $_i$  ( $i \geq 1$ ) con  $T_i$  calculado por la compuerta combinacional de arriba (un AND de  $i$  entradas para “todos en 1” y otro AND de  $i$  entradas negadas para “todos en 0”, seleccionados por  $UD/UD'$  con dos AND adicionales y un OR final). Reloj común a los 4 FF (contador **sincrónico**, no ripple). Reset asincrónico común (Clear) en los 4 FF.

## Ejercicio 2 ★ — Verificador secuencial de paridad

**Enunciado:** 1 bit de entrada, 1 bit de salida; la salida vale 1 si la cantidad de unos recibidos hasta el momento (incluyendo el bit actual) es par.

**Diseño:** 1 FF D. Definimos  $Q$  = “la cantidad de unos vista hasta ahora es par” ( $Q=1 \Leftrightarrow$  par). Al llegar un nuevo bit  $e$ , la paridad se invierte si  $e=1$  y se mantiene si  $e=0$ :

$$D = Q \oplus e$$

Como la salida debe reflejar la paridad **incluyendo el bit que acaba de llegar** (no la de un ciclo atrás), la salida se calcula combinatorialmente con la misma expresión que  $D$  — es una máquina de **Mealy** (la salida depende de estado y entrada actual simultáneamente, disponible en el mismo ciclo, sin esperar el flanco de reloj):

$$S = Q \oplus e \quad ; (=D)$$

Reset activo por nivel bajo → Preset asincrónico que fuerza  $Q=1$  (0 unos recibidos = paridad par).

**Verificación (Python):** se simularon 2000 cadenas aleatorias de bits (largo 1 a 20), contando manualmente la cantidad de unos acumulada y comparando la paridad esperada contra la salida del circuito simulado bit a bit ( $D=Q \oplus e$ ,  $S=D$ , arrancando en  $Q=1$ ). Coincidencia exacta en las 2000 cadenas.

**Circuito:** 1 FF D.  $D = \text{XOR}(Q, e)$  (una compuerta XOR, o 2 AND + 1 OR + 2 NOT si no se dispone de XOR directa). Salida  $S$  tomada directamente de la salida de esa misma compuerta XOR (no de  $Q$ ). Preset asincrónico activado por Reset (nivel bajo).

### Ejercicio 3 ★ — Comparador serial de bits

**Enunciado:** FF tipo D; entra una cadena de bits; la salida es 1 si el bit actual es igual al bit inmediatamente anterior. Reset activa en 1.

**Diseño:** 1 FF D que simplemente “recuerda” el bit anterior.

$$D = e \quad (\text{el bit actual pasa a ser “el anterior” para el próximo ciclo})$$

$$S = \overline{Q \oplus e} = \text{XNOR}(Q, e)$$

donde Q guarda el valor del bit anterior. Reset (nivel alto, según letra) → Clear asincrónico, fuerza  $Q=0$  (convención: antes del primer bit, “el anterior” se considera 0).

**Verificación (Python):** 2000 cadenas aleatorias; para cada bit se comparó contra el bit inmediatamente anterior de la propia cadena (o 0 para el primer bit, por la convención de reset) y se contrastó contra  $\text{XNOR}(Q, e)$  simulado. Coincidencia exacta.

**Circuito:** 1 FF D, entrada  $D=e$ . Salida  $S$  = compuerta XNOR entre Q y e (2 AND + 1 OR + 2 NOT, o XNOR directa). Clear asincrónico activo en 1.

### Ejercicio 4 ★★ — Contador de código Gray de 3 bits

**Enunciado:** FF tipo D; secuencia 000, 001, 011, 010, 110, 111, 101, 100, vuelve a 000; entrada de reset que pone el contador en 000.

**Tabla de transiciones** (verificada exhaustivamente contra la secuencia dada):

| $q_2q_1q_0$ | próximo estado | $D_2$ | $D_1$ | $D_0$ |
|-------------|----------------|-------|-------|-------|
| 000         | 001            | 0     | 0     | 1     |
| 001         | 011            | 0     | 1     | 1     |
| 011         | 010            | 0     | 1     | 0     |
| 010         | 110            | 1     | 1     | 0     |
| 110         | 111            | 1     | 1     | 1     |
| 111         | 101            | 1     | 0     | 1     |
| 101         | 100            | 1     | 0     | 0     |
| 100         | 000            | 0     | 0     | 0     |

**Ecuaciones minimizadas (Karnaugh de 3 variables, verificadas por SOP exhaustivo con sympy):**

$$D_2 = q_0 q_2 + q_1 q_0' \quad D_1 = q_0 q_2' + q_1 q_0' \quad D_0 = q_1 q_2 + q_1' q_2'$$

**Verificación (Python/sympy):** se generó SOPform a partir de las 8 filas de la tabla (sin asumir ninguna fórmula de antemano) y se comprobó que reproduce exactamente las 8 filas — coincide con las ecuaciones de la Guía Temática M4 (sección 6.5), confirmadas ahora también por minimización independiente.

**Chequeo manual de una fila:** estado 011 ( $q_2=0, q_1=1, q_0=1$ ).  $D_2=q_0q_2+q_1q_0'=1\cdot 0+1\cdot 0=0$ .  
 $D_1=q_0q_2'+q_1q_0'=1\cdot 1+1\cdot 0=1$ .  $D_0=q_1q_2+q_1'q_2'=1\cdot 0+0\cdot 1=0$ . Resultado  $D_2D_1D_0=010 \rightarrow$  coincide con la tabla (011 $\rightarrow$ 010) ✓.

Reset activo por nivel bajo  $\rightarrow$  Clear asincrónico en los 3 FF (000 es la combinación “todo cero”, compatible con un Clear estándar).

## Ejercicio 5 ★★ — Sumador serial de 8 bits

**Enunciado:** FF tipo D + un sumador; suma los 4 bits menos significativos de la entrada con los 4 bits más significativos; resultado en 4 bits paralelo; reset activa en 1.

**Idea:** la entrada de 8 bits ( $a_7\dots a_0$ ) llega **en serie**, un bit por flanco de reloj. Se procesan 4 flancos: en el flanco  $k$  se suman el bit  $k$  del “número bajo” ( $a_{0..3}$ ) con el bit  $k$  del “número alto” ( $a_{4..7}$ ), más el acarreo guardado del flanco anterior, usando **un único sumador completo de 1 bit** reutilizado. El acarreo se guarda en 1 FF D (realimentado a la entrada “carry” del sumador); cada bit de suma resultante se guarda en uno de 4 FF D de salida (registro de salida paralelo).

| Flanco | Entradas al sumador             | Guarda                                                               |
|--------|---------------------------------|----------------------------------------------------------------------|
| 1      | $a_4, a_0$ , carry=0 (reset)    | $S_0$ en FF de salida 0; carry en FF de acarreo                      |
| 2      | $a_5, a_1$ , carry (del paso 1) | $S_1$ ; nuevo carry                                                  |
| 3      | $a_6, a_2$ , carry (del paso 2) | $S_2$ ; nuevo carry                                                  |
| 4      | $a_7, a_3$ , carry (del paso 3) | $S_3$ ; carry final (se descarta — el resultado pedido es de 4 bits) |

Reset (activo en 1, según letra)  $\rightarrow$  Clear asincrónico sobre el FF de acarreo (arranca en carry=0) y sobre los 4 FF de salida.

**Verificación (Python):** 3000 valores aleatorios de 8 bits; para cada uno se calculó  $(a_{4..7} + a_{0..3}) \bmod 16$  “a mano” y se comparó contra la simulación ciclo a ciclo del sumador completo + FF de acarreo realimentado, sobre 4 flancos. Coincidencia exacta en los 3000 casos.

**⚠ Nota de transparencia:** la letra pide el resultado en “cuatro bits en paralelo” — se interpretó que el acarreo de salida del último flanco (posible overflow de la suma de dos números de 4 bits, rango 0-30) se descarta y no forma parte del resultado pedido, ya que no se menciona un quinto bit ni una señal de overflow. Si el enunciado real (o la corrección del docente) esperara conservar ese acarreo final como un bit adicional, bastaría con agregar un FF D más para guardarlo — el resto del diseño no cambia.

## Ejercicio 6 ★ (OpenFing, sin solución en el material del curso) — Contador de números primos

**Enunciado:** registro contador que recorre únicamente los primos entre 0 y 15, en el orden 2, 3, 5, 7, 11, 13, y vuelve a 2.

**Diseño:** 4 bits de estado ( $q_3q_2q_1q_0$ ), usando directamente el valor binario natural de cada primo como código de estado ( $2=0010$ ,  $3=0011$ ,  $5=0101$ ,  $7=0111$ ,  $11=1011$ ,  $13=1101$ ). De los 16 códigos posibles, solo 6 son estados válidos — los 10 restantes ( $0,1,4,6,8,9,10,12,14,15$ ) **nunca ocurren** en operación normal y se tratan como *don't care* en el Karnaugh (Guía M4, sección 6.6).

**Tabla de transiciones (los 6 estados válidos):**

| Estado (dec) | $q_3q_2q_1q_0$ | próximo | $D_3D_2D_1D_0$ |
|--------------|----------------|---------|----------------|
| 2            | 0010           | 3       | 0011           |
| 3            | 0011           | 5       | 0101           |
| 5            | 0101           | 7       | 0111           |
| 7            | 0111           | 11      | 1011           |
| 11           | 1011           | 13      | 1101           |
| 13           | 1101           | 2       | 0010           |

**Ecuaciones minimizadas (Karnaugh de 4 variables con don't-cares, verificadas por SOP exhaustivo con sympy usando los 10 códigos no usados como comodines):**

$$\begin{aligned} D_3 &= q_1 q_2 + q_3 q_2' \\ D_2 &= q_0 q_2' + q_1' q_3' \\ D_1 &= q_2 + q_0' \\ D_0 &= q_2' + q_3' \end{aligned}$$

**Verificación (Python/sympy):** se construyó la tabla de verdad completa de 16 filas (6 filas con salida fija según la tabla de arriba, 10 filas marcadas como don't care), se minimizó con SOPform aprovechando los comodines, y se comprobó que las 4 ecuaciones reproducen exactamente el ciclo  $2 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 11 \rightarrow 13 \rightarrow 2$  en las 6 filas válidas (las 10 filas de don't care, por construcción, no importan qué valor tomen — nunca se alcanzan si el reset y las ecuaciones son correctos).

**Reset:** no se especifica nivel en la letra  $\rightarrow$  default de la letra general: **nivel bajo**. Como el estado de reposo “todo cero” (0000) **no** es un estado válido del ciclo, el reset no puede ser un Clear estándar en las 4 líneas: hay que llevar el contador al primer primo ( $2 = 0010$ ). Eso se logra con Clear asincrónico (fuerza 0) en  $q_3$ ,  $q_2$ ,  $q_0$ , y **Preset** asincrónico (fuerza 1) en  $q_1$ , todos disparados por la misma señal de reset.

## Ejercicio 7 ★★★ (examen 26/07/2014, sin solución en el material del curso) — Cronómetro minutos:segundos

**Enunciado:** cronómetro 00:00 a 59:59 en 4 displays de 7 segmentos (MM:SS), reloj de 1Hz, botón reset ( $\rightarrow 00:00$ ) y botón incrementar (mientras se mantiene presionado, suma 1 minuto por cada segundo que dure la pulsación, sin que los segundos avancen durante ese tiempo).

**⚠ Nota de transparencia:** este es el único ejercicio ★★★ del práctico y no tiene solución en el material del curso (es un ejercicio de examen real). No hay una tabla de estados “chica” para verificar contra el enunciado como en los ejercicios anteriores — es un problema de **integración de varios contadores en cascada**. Se explica acá el método de diseño completo, con las ecuaciones de cada etapa derivables por Karnaugh estándar (mismo método del Ej. 1 y de la Guía M4 sección 6.2), y se verificó por **simulación de sistema completo** (no

gate-level) que la lógica de acarreo entre etapas y la lógica del botón incrementar son consistentes — no se dibujó el circuito compuerta por compuerta completo de las ~14 Flip-Flops resultantes por ser un desarrollo demasiado extenso para reproducir a mano; se da el patrón exacto para derivar cada etapa (idéntico al Ej.1/6).

## PASO 1 — DESCOMPOSICIÓN EN 4 CONTADORES EN CASCADA

Un reloj MM:SS no es un único contador binario: es 4 “dígitos” independientes, cada uno un contador módulo distinto, con acarreo entre ellos (igual que un reloj analógico digital normal):

| Etapas                  | Rango | Módulo | Bits necesarios |
|-------------------------|-------|--------|-----------------|
| SU (segundos, unidades) | 0-9   | 10     | 4 (0000-1001)   |
| ST (segundos, decenas)  | 0-5   | 6      | 3 (000-101)     |
| MU (minutos, unidades)  | 0-9   | 10     | 4               |
| MT (minutos, decenas)   | 0-5   | 6      | 3               |

Total: 14 Flip-Flops D.

## PASO 2 — LÓGICA DE ACARREO ENTRE ETAPAS (CONTADOR NORMAL, INCR=0)

Cada etapa avanza en el mismo flanco de reloj (1Hz) que las demás, pero solo “realmente” cambia cuando le llega **acarreo** de la etapa anterior (excepto SU, que avanza siempre en cada flanco si no se está incrementando):

- SU cuenta 0→9→0 en cada flanco de 1Hz (si INCR=0). Genera **acarreo C1=1** exactamente cuando SU=9 en ese flanco (va a hacer 9→0).
- ST solo avanza si recibe C1=1 (contador con habilitación, mismo patrón de MUX que “carga paralela” del Ej. de la Guía M4 sección 6.4:  $D_{ST} = C1 \cdot (ST+1 \bmod 6) + C1' \cdot ST$ ). Genera **acarreo C2=1** cuando (C1=1 y ST=5) — va a hacer 5→0.
- MU solo avanza si recibe C2=1 (mismo patrón). Genera **acarreo C3=1** cuando (C2=1 y MU=9).
- MT solo avanza si recibe C3=1.

Esta es exactamente la técnica de “contador con habilitación” (variante del Ej. 1 con carga paralela de la Guía M4 6.4, pero habilitando en vez de cargar un dato externo):  $D_i = EN \cdot (\text{lógica de contar})_i + EN' \cdot q_i$  (si EN=0, el FF simplemente retiene su valor — no cuenta ese flanco).

## PASO 3 — BOTÓN INCREMENTAR (INCR)

Mientras INCR=1: el pulso de 1Hz se **desvía** de SU hacia MU directamente (SU y ST quedan congelados, con EN=0 para ambos), y MU/MT cuentan con la misma lógica de acarreo del Paso 2 pero alimentados por ese mismo pulso. Esto es un multiplexor de “quién recibe el pulso de reloj efectivo”: si INCR=0 el pulso habilita la cadena de segundos (con su acarreo natural hacia minutos); si INCR=1 el pulso habilita directamente la cadena de minutos, sin tocar segundos.

## PASO 4 — RESET

Botón de reset (nivel bajo, default de la letra) → Clear asincrónico simultáneo en los 14 FF → SU=ST=MU=MT=0 → display 00:00.

## PASO 5 — DISPLAYS DE 7 SEGMENTOS

Cada uno de los 4 dígitos (SU, ST, MU, MT) alimenta un decodificador combinacional BCD→7 segmentos independiente (el bloque de la Figura 2 del enunciado) — es un circuito puramente combinacional (una ROM/decodificador, técnica del Práctico 4/M3), sin FF adicionales: los 4 bits de cada contador van directo a las entradas D,C,B,A del decodificador correspondiente.

**Verificación (Python, simulación de sistema completo):** se implementó la cascada SU→ST→MU→MT con la lógica de acarreo del Paso 2 y el desvío del Paso 3, y se corrieron 3 pruebas:

- 3661 pulsos normales (INCR=0) desde 00:00 → dio exactamente 01:01 (3661 s = 61 min 1 s = 1:01 en el reloj de 60 minutos), coincidiendo con el cálculo aritmético independiente (3661 mod 3600, convertido a mm:ss).
- 5 pulsos con INCR=1 desde 00:00 → dio 05:00 (segundos sin tocar, 5 minutos sumados) ✓.
- 30 s normales seguidos de 35 pulsos con INCR=1 → dio 35:30 (segundos se mantienen en 30, minutos suben en 35 con acarreo de decena incluido: 3 decenas + 5 unidades) ✓, confirmando que el acarreo MU→MT funciona igual estando en modo INCR.

Las 3 pruebas coincidieron exactamente con el resultado esperado calculado de forma independiente (aritmética de reloj, no derivada del mismo código de simulación).

## Resumen de técnicas usadas en este práctico

| Ejercicio      | Técnica                                                                                                                                                                                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Teóricas (a-e) | Nivel vs. flanco, RAM=secuencial, igualar ecuaciones características (D→JK), R-S como demostración de “misma entrada, dos salidas posibles”, asincronismo de Reset/Preset                                                                                                       |
| 1              | Contador up/down con FF T (fórmula de acarreo/préstamo generalizable a n bits); alternativa D vía Karnaugh de 5 variables                                                                                                                                                       |
| 2              | Máquina de Mealy de 1 estado-bit; salida = misma expresión que la entrada del FF (disponible sin esperar flanco)                                                                                                                                                                |
| 3              | FF D como “memoria de 1 bit desplazada”; comparación vía XNOR                                                                                                                                                                                                                   |
| 4              | Contador de secuencia arbitraria (no binaria): tabla de transiciones ad-hoc + Karnaugh estándar                                                                                                                                                                                 |
| 5              | Arquitectura serial: 1 sumador completo reutilizado + FF de acarreo realimentado + registro de salida paralelo                                                                                                                                                                  |
| 6              | Contador de subconjunto de estados con codificación = valor numérico + don't cares en el Karnaugh + reset con Clear/Preset mixto (el estado de reposo no es “todo cero”)                                                                                                        |
| 7              | Descomposición de un sistema grande en contadores módulo-N en cascada con señales de acarreo/habilitación (mismo patrón que “carga paralela” de la Guía M4, generalizado a “habilitación”) + multiplexado de a qué cadena se dirige el pulso de reloj según un botón de control |

**Para el examen:** el patrón más transferible de este práctico es que **casi todo diseño de contador “raro” (up/down, código no binario, subconjunto de estados, sistema con varios dígitos) se resuelve con el mismo método de 6 pasos de la Guía M4** (tabla de estados → codificación → tabla de transiciones → ecuación característica del FF → Karnaugh, con don't-cares si sobran códigos → circuito), y que la **habilitación/carga condi-**

**cional** (Ej. 1 con UD, Ej. 7 con el botón incrementar) siempre se resuelve viendo cada  $D_i$  como un multiplexor: “si la condición de control se cumple, avanza la lógica de contar; si no, retiene el valor actual” — es exactamente la misma idea que un contador con carga paralela, aplicada a “quién recibe el pulso” en vez de “qué dato paralelo se carga”. El reset merece atención especial: releer siempre si la letra dice explícitamente “activa en 1” (nivel alto) o si aplica el default de nivel bajo, y verificar si el estado de reposo deseado es realmente “todo cero” (Clear simple) o requiere una combinación mixta de Clear/Preset por bit (Ej. 6).